

Efficient Resource Optimization for Split Federated Learning

Wei Wei, *Graduate Student Member, IEEE* and Xianhao Chen, *Member, IEEE*

Abstract—Split federated learning (SFL) has emerged as a powerful paradigm for model training at the edge. However, SFL inherently involves discrete decision variables for model splitting and resource allocation, resulting in a challenging mixed-integer problem. Consequently, prior optimization schemes for SFL are either *heuristic* or *computationally inefficient*, which cannot handle large-scale user populations. To address this limitation, this work establishes an efficient optimization framework for SFL under resource-constrained networks. Our framework jointly optimizes model splitting and resource allocation to minimize training cost, which is defined as the weighted sum of latency and energy costs. We first study the model splitting problem and develop a polynomial-time algorithm that achieves the global optimum. Then, we extend the approach to the joint model splitting and resource allocation problem. In this case, we formulate it as a two-dimensional master problem and develop an efficient approximation method with a $(1 + \epsilon)$ -approximation guarantee. Extensive experiments show that the proposed approach provides efficient solutions to strike the optimal energy–latency tradeoff.

Index Terms—Mobile edge computing (MEC), split federated learning, model splitting, resource allocation.

I. INTRODUCTION

Edge learning enables artificial intelligence (AI) computation close to the data source, enhancing privacy, saving bandwidth, and enabling personalization of edge devices [1]–[5]. However, the rapid expansion of AI models presents unprecedented computational challenges. State-of-the-art learning is inherently incompatible with the limited hardware resources of edge devices. For instance, even powerful edge devices such as the NVIDIA Jetson AGX Xavier struggle to train models with more than 50 million parameters [6]. Given these challenges, split federated learning (SFL) [7]–[9], which utilizes a client-server architecture, presents a promising solution. In SFL, the AI model is divided into two segments: the initial layers are trained on clients over local data, whereas the subsequent layers are trained on a server that provides more powerful computing resources. Moreover, a federated server (or simply Fed server) periodically aggregates client-side sub-models, akin to the federated learning (FL) paradigm [10]–[12], to synchronize models across multiple clients in parallel. SFL alleviates the hardware constraints of clients by offloading the most demanding computations to more powerful servers, thereby recognized as a promising framework to enable large-scale model training on edge devices.

The work was supported in part by the Research Grants Council of Hong Kong under Grant 27213824 and CRS HKU702/24. (Corresponding author: Xianhao Chen)

Wei Wei and Xianhao Chen are with the Department of Electrical and Computer Engineering, the University of Hong Kong, Pok Fu Lam, Hong Kong SAR, 999077. (e-mail: weiwei@eee.hku.hk; xchen@eee.hku.hk).

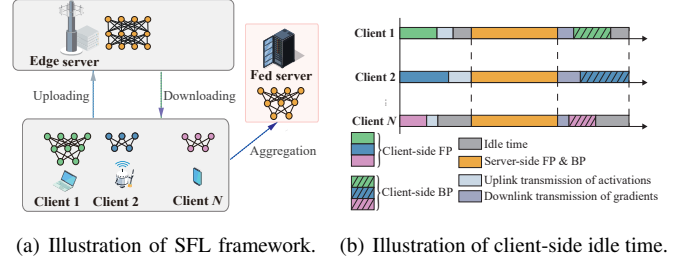


Fig. 1. Illustration of SFL workflow. (a) SFL scenario includes uploading, downloading and aggregation processes. (b) Since the server starts only after all clients finish local FP and activation uploading, and the next round begins only after all clients complete BP and gradient downloading, moderately reducing the GPU frequency and transmission power of faster clients does not affect the overall training time but can save energy.

As with other edge learning paradigms, resource optimization is essential for SFL, since its performance critically depends on how computation, communication, and model-splitting decisions are orchestrated under limited network and computing resources [13]–[16]. On the client side, heterogeneous communication and computing capabilities, combined with model-splitting choices, often create a straggler bottleneck where the slowest clients dominate the overall training time (see Fig. 1(b)). Mitigating this requires strategic resource management. First, *optimizing power and computing resource allocation* for heterogeneous clients can save energy while prioritizing resources for slower clients to accelerate training. Second, the *model splitting decision* determines how computation and communication loads are distributed between the client and server, thereby affecting both energy consumption and end-to-end latency, as shown in Fig. 2. Thus, realizing efficient SFL at the edge requires the joint optimization of computation, communication, and model-splitting decisions.

In recent years, the SFL literature has addressed resource allocation for improving training efficiency, covering client/server workload coordination [17]–[20], model splitting [13], [14], [21], [22], wireless resource allocation [22]–[26], and aggregation control [13], [14]. While these advances have significantly improved SFL performance, two fundamental challenges in SFL remain unresolved: First, although the model-splitting decision is a core component of SFL, existing studies still lack a scalable polynomial-time solution. It remains an open question whether cut-layer optimization in SFL, due to its combinatorial structure, can admit an exact polynomial-time algorithm. Resolving this problem is crucial for enabling scalable SFL optimization with large user populations (e.g., on the order of hundreds of users or more). Second, beyond model partitioning, the performance

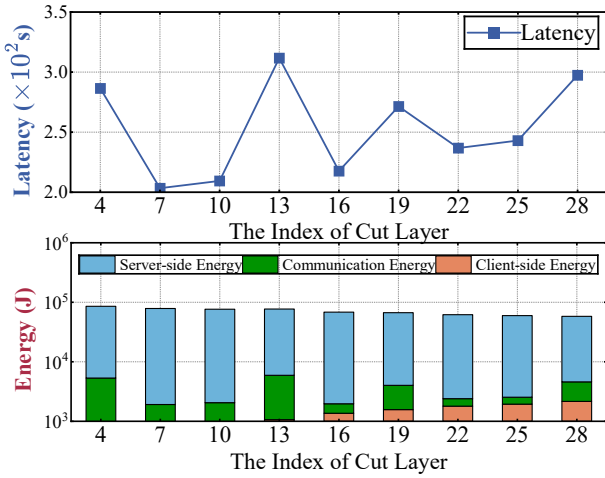


Fig. 2. Impact of cut layer on energy and latency. The experiments are conducted with ResNet-50 on the CIFAR-10 dataset under non-IID settings. Other configurations are identical to experimental settings in Section VII.

of SFL is strongly influenced by GPU operating frequencies and transmission powers, which jointly determine the energy–latency trade-off in heterogeneous edge environments. Nevertheless, the joint optimization of model partitioning and resource allocation substantially increases the complexity of the problem, rendering the overall optimization task more challenging.

Motivated by these challenges, this paper aims to answer two research questions:

- Q1:** Can model-splitting optimization in SFL be solved optimally in polynomial time?
- Q2:** Can model-splitting decisions, GPU frequency scaling, and transmission power control be jointly optimized within a unified framework?

This paper firmly answers both questions through the following contributions.

- We establish a unified resource optimization framework for SFL by jointly considering model splitting, GPU frequency scaling, and transmission power control.
- For the model-splitting problem, we derive an optimal polynomial-time solution approach.
- For the joint model-splitting and resource optimization problem, we formulate it as a two-dimensional master problem and develop a polynomial-time approximation scheme with a $(1 + \epsilon)$ -approximation guarantee.
- Extensive experiments demonstrate that the proposed method consistently outperforms baseline schemes in terms of both latency–energy tradeoff and running-time efficiency.

The remainder of this paper is organized as follows. Section II reviews related work. Section III introduces the system model and the proposed resource optimization framework for SFL. Section IV presents the problem formulation. Section V considers a special case with model-splitting optimization and derives a polynomial-time optimal solution. Section VI investigates the general joint optimization problem, reformulates it as a two-dimensional master problem, and develops an

approximation algorithm. Section VII presents the numerical results. Finally, Section VIII concludes this paper.

II. RELATED WORK

Resource management in distributed learning and edge AI. Recent studies have shown that runtime resource control, especially GPU frequency scaling and power management, can substantially affect the time and performance of deep learning. For example, Drechsler et al. [27] show that lowering GPU frequency can significantly reduce power consumption for CNN inference with only marginal impact on computation time. More recent studies extend this observation to training tasks. Maliakel et al. [28] systematically analyze the impact of dynamic voltage and frequency scaling across different models and tasks, while throttLL’eM [29] exploits GPU frequency scaling to improve training efficiency under latency constraints. Beyond frequency control, several works have explored other forms of runtime resource control for improving the time–resource tradeoff of deep learning systems. Zeus [30] jointly controls batch size and GPU power limits for recurring DNN training jobs to better balance training time and energy consumption. EnvPipe [31] improves training efficiency by leveraging idle pipeline intervals and dynamically adjusting streaming multiprocessor frequencies without sacrificing performance. PCCL [32] focuses on distributed training and identifies more resource-efficient GPU frequency settings for collective communication calls, thereby reducing communication overhead with negligible throughput degradation. Perseus [33] further improves the time–resource tradeoff of large-model training by optimizing computation scheduling along the performance frontier.

These studies demonstrate that resource control is an effective mechanism for improving the efficiency of deep learning workloads in both distributed learning and edge AI systems. However, they are primarily designed for conventional training, inference, or distributed GPU platforms. Thus, they either rely on fixed model partitioning or fail to address partitioning across heterogeneous edge devices.

Resource management in SFL. Recent studies on SFL have increasingly focused on resource allocation for improving training efficiency in heterogeneous edge environments. For example, ESFL [17] optimizes client-side workload distribution and server-side resource allocation over heterogeneous wireless devices, while Wu *et al.* [23] and HMSFL [21] further incorporate model splitting, communication/computing resource allocation, and hierarchical split designs to improve training efficiency. In addition, AdaptSFL and HSFL [13], [14] show that split decisions and aggregation intervals can be adjusted to better balance communication and computation during training, thereby affecting communication overhead, local computation burden, and convergence speed. More recently, ASFL [22] combines adaptive model splitting with wireless resource allocation, and Wang *et al.* [24] further consider resource-constrained edge environments with unreliable wireless links. Nevertheless, most existing approaches focus on latency reduction, without considering a unified latency–energy tradeoff. More importantly, the joint optimization of

TABLE I
SUMMARY OF IMPORTANT NOTATIONS.

Notation	Description
$\mathcal{N}$	The set of all clients
N	The number of clients
L	The number of layers in the global model
$L_{c,i}$	The cut layer of client i
n_i	The number of GPU FLOPs per cycle of client i
n_S	The number of GPU FLOPs per cycle of the edge server
$f_{i,j}$	The clock frequency of client i 's GPU
$f_{S,j}$	The clock frequency of the edge server's GPU
$a_i(L_{c,i})$	The activations of client i at the $L_{c,i}$ -th cut layer
ρ_l	The FP computational cost of the l -th DNN layer
ϖ_l	The BP computational cost of the l -th DNN layer
B_i^U	The uplink bandwidth allocated to client i
B_i^D	The downlink bandwidth allocated to client i
$P_{i,j}^{\text{TX}}$	The transmit power of client i at the j -th stage
$P_{S,j}^{\text{TX}}$	The transmit power of the edge server at the j -th stage
h_i	The uplink channel fading coefficient
h_S	The downlink channel fading coefficient
N_0	The noise power spectral density
$g_i(L_{c,i})$	The gradients of client i at the $L_{c,i}$ -th cut layer
G_i	The coefficient depends on the chip of client i
G_S	The coefficient depends on the chip of the edge server
$u_{i,j}$	The control input of client i at the j -th stage
λ	Balances energy efficiency and training latency

model splitting and resource allocation remains largely open, where polynomial-time algorithms with provable approximation guarantees are still lacking.

III. SYSTEM MODEL

A. SFL Architecture

We consider an SFL system with a set of clients $\mathcal{N} = \{1, \dots, N\}$, an edge server that executes the server-side sub-model, and a federated server that periodically aggregates model parameters. For each client i , the global DNN model is partitioned into a *client-side sub-model*, which comprises the initial DNN layers up to the cut layer indexed by $L_{c,i}$, and a *server-side sub-model*, which includes the remaining DNN layers from $L_{c,i} + 1$ to the final output layer. Client i performs local forward pass (FP) up to the cut layer and transmits the corresponding intermediate activations to the edge server, which continues the remaining computation and returns the cut-layer gradients to the client for completing backward pass (BP).

B. Stages of SFL

Within one SFL round, each client i performs a five-stage pipeline¹ that captures both computation and communication

¹After every I local training rounds, the federated server performs model aggregation to update the global parameters across all clients. The aggregation process is executed on a much slower timescale than the intra-round SFL pipeline. Thus, aggregation time can be appended as a constant offset at the end of I local training rounds and the resource optimization of model aggregation is omitted in this paper.

operations. Specifically, the stages include: 1) client-side FP, 2) uplink transmissions of activations, 3) server-side FP and BP, 4) downlink transmissions of gradients, and 5) client-side BP. We denote these stages as $\mathcal{J}_i^{\text{FP}}$, $\mathcal{J}_i^U$, $\mathcal{J}_i^{\text{FP+BP}}$, $\mathcal{J}_i^D$, and $\mathcal{J}_i^{\text{BP}}$, respectively.

- 1) **Client-side FP.** Client i executes the first $L_{c,i}$ DNN layers on its local data, producing activations $a_i(L_{c,i})$ at the cut layer. The FP time of client i is modeled as

$$s_{i,j}(f_{i,j}, L_{c,i}) = \sum_{l=1}^{L_{c,i}} \frac{\rho_l}{f_{i,j} n_i}, \quad j = 1, \quad (1)$$

where ρ_l denotes the FP computational cost (in FLOPs) of the l -th DNN layer, n_i denotes the number of GPU FLOPs per cycle and $f_{i,j}$ represents the clock frequency of client i 's GPU.

- 2) **Uplink transmissions of activations.** Client i transmits the activations $a_i(L_{c,i})$ to the edge server. The data size of $a_i(L_{c,i})$ is denoted by $d_i^U(L_{c,i}) = \text{bits}(a_i(L_{c,i}))$, and the corresponding transmission time is given by

$$s_{i,j}(P_{i,j}^{\text{TX}}, L_{c,i}) = \frac{d_i^U(L_{c,i})}{B_i^U \log \left(1 + \frac{P_{i,j}^{\text{TX}} |h_i|^2}{B_i^U N_0} \right)}, \quad j = 2. \quad (2)$$

Here, B_i^U denotes the uplink bandwidth allocated to client i , $P_{i,j}^{\text{TX}}$ is the transmit power of client i at the j -th stage, h_i denotes the uplink channel fading coefficient, h_i^2 is the corresponding channel gain, and N_0 is the noise power spectral density.

- 3) **Server-side FP and BP.** Upon receiving $a_i(L_{c,i})$, the edge server performs the remaining forward and backward passes from layer $L_{c,i} + 1$ to L , and outputs the corresponding gradient $g_i(L_{c,i})$. The data size of $g_i(L_{c,i})$ is denoted by $d_i^D(L_{c,i}) = \text{bits}(g_i(L_{c,i}))$. The server-side computation time is

$$s_{i,j}(f_{S,j}, L_{c,i}) = \sum_{l=L_{c,i}+1}^L \frac{\rho_l + \varpi_l}{f_{S,j} n_S}, \quad j = 3, \quad (3)$$

where ϖ_l denotes the BP computational cost (in FLOPs) of the l -th DNN layer, $f_{S,j}$ is the clock frequency of the edge server's GPU, and n_S denotes the number of GPU FLOPs per cycle of edge server.

- 4) **Downlink transmissions of gradients.** The edge server transmits $g_i(L_{c,i})$ back to client i through the downlink channel. The data size of $g_i(L_{c,i})$ is denoted by $d_i^D(L_{c,i})$, and the corresponding transmission time is modeled as

$$s_{i,j}(P_{S,j}^{\text{TX}}, L_{c,i}) = \frac{d_i^D(L_{c,i})}{B_i^D \log \left(1 + \frac{P_{S,j}^{\text{TX}} |h_S|^2}{B_i^D N_0} \right)}, \quad j = 4, \quad (4)$$

Here, B_i^D denotes the downlink bandwidth, $P_{S,j}^{\text{TX}}$ is the transmit power of the edge server at the j -th stage, and h_S represents the channel fading coefficient.

- 5) **Client-side BP.** After receiving the gradients from the edge server, client i performs BP on its local DNN layers, and subsequently updates the parameters of its

client-side sub-model. The service time for this stage is modeled as

$$s_{i,j}(f_{i,j}, L_{c,i}) = \sum_{l=1}^{L_{c,i}} \frac{\varpi_l}{f_{i,j} n_i}, \quad j = 5. \quad (5)$$

C. Per-round Latency

Based on the defined five-stage SFL, we consider a *parallel execution model* across clients within each SFL round. Under this model, the per-round training latency of client-side FP and uplink is dominated by the slowest client. Moreover, the edge server maintains a shared server-side sub-model and must process the server-side FP and BP for all clients sequentially. Thus, server-side workloads contribute additively, yielding an aggregate latency term. Similarly, the per-round training latency of client-side BP and downlink is dominated by the slowest client. Accordingly, we define the per-round latency as

$$\begin{aligned} T_r(f_{i,j}, P_{i,j}^{\text{TX}}, f_{S,j}, P_{S,j}^{\text{TX}}, L_{c,i}) = \\ \max_{i \in \mathcal{N}} \left\{ s_{i,1}(f_{i,1}, L_{c,i}) + s_{i,2}(P_{i,2}^{\text{TX}}, L_{c,i}) \right\} + \sum_{i \in \mathcal{N}} s_{i,3}(f_{S,3}, L_{c,i}) \\ + \max_{i \in \mathcal{N}} \left\{ s_{i,4}(P_{S,4}^{\text{TX}}, L_{c,i}) + s_{i,5}(f_{i,5}, L_{c,i}) \right\}. \end{aligned} \quad (6)$$

D. Energy Model

We next quantify the per-round energy consumption by incorporating both computing and communication energy in the five-stage SFL pipeline. For each client $i \in \mathcal{N}$, we define the energy consumed at the j -th stage ($j \in \{1, \dots, 5\}$) under cut layer $L_{c,i}$ as

$$E_{i,j}(f_{i,j}, P_{i,j}^{\text{TX}}, f_{S,j}, P_{S,j}^{\text{TX}}, L_{c,i}) \triangleq \begin{cases} \frac{G_i}{n_i^3} \frac{\left(\sum_{l=1}^{L_{c,i}} \rho_l\right)^3}{[s_{i,j}(f_{i,j}, L_{c,i})]^2}, & j = 1, \\ \frac{B^U s_{i,j}(P_{i,j}^{\text{TX}}, L_{c,i}) N_0}{h_i^2} \left(2^{\frac{d_i^U(L_{c,i})}{B^U s_{i,j}(P_{i,j}^{\text{TX}}, L_{c,i})}} - 1 \right), & j = 2, \\ \frac{G_S}{n_S^3} \frac{\left(\sum_{l=L_{c,i}+1}^L (\rho_l + \varpi_l)\right)^3}{[s_{i,j}(f_{S,j}, L_{c,i})]^2}, & j = 3 \\ \frac{B^D s_{i,j}(P_{S,j}^{\text{TX}}, L_{c,i}) N_0}{h_S^2} \cdot \left(2^{\frac{d_i^D(L_{c,i})}{B^D s_{i,j}(P_{S,j}^{\text{TX}}, L_{c,i})}} - 1 \right), & j = 4, \\ \frac{G_i}{n_i^3} \frac{\left(\sum_{l=1}^{L_{c,i}} \varpi_l\right)^3}{[s_{i,j}(f_{i,j}, L_{c,i})]^2}, & j = 5, \end{cases} \quad (7)$$

where the coefficients G_i and G_S [in Watt/(cycle/s)³] depend on the chip architecture of client i and the edge server [34]. Accordingly, the total energy consumption of one SFL round is given by

$$E_r(f_{i,j}, P_{i,j}^{\text{TX}}, f_{S,j}, P_{S,j}^{\text{TX}}, L_{c,i}) =$$

$$\sum_{i \in \mathcal{N}} \sum_{j=1}^5 E_{i,j}(f_{i,j}, P_{i,j}^{\text{TX}}, f_{S,j}, P_{S,j}^{\text{TX}}, L_{c,i}). \quad (8)$$

E. Control Input of Each SFL Stage

The control input $u_{i,j}(t)$ represents the adjustable resource variables that govern the evolution rate $f_{i,j}(z_{i,j}(t), u_{i,j}(t))$ and thus determine the stage service time $s_{i,j}(u_{i,j}(t), L_{c,i})$. Formally,

$$u_{i,j}(t) = \begin{cases} f_{i,j}(t), & j \in \{1, 5\}, \\ f_{S,j}(t), & j = 3, \\ P_{i,j}^{\text{TX}}(t), & j = 2, \\ P_{S,j}^{\text{TX}}(t), & j = 4. \end{cases} \quad (9)$$

Here, $f_{i,j}(t)$ and $f_{S,j}(t)$ denote the instantaneous GPU frequencies of client i and the edge server, while $P_{i,j}^{\text{TX}}(t)$ and $P_{S,j}^{\text{TX}}(t)$ represent their respective uplink and downlink transmit powers. The cut-layer index $L_{c,i}$ specifies which subset of DNN layers is executed locally and which subset is offloaded to the edge server, thereby influencing the mapping from control inputs to the service time $s_{i,j}(\cdot)$. Moreover, each client and the edge server are subject to bounded GPU frequencies and transmit powers, $f_i^{\min} \leq f_{i,j} \leq f_i^{\max}$, $f_S^{\min} \leq f_{S,j} \leq f_S^{\max}$, $P_{i,j}^{\text{TX},\min} \leq P_{i,j}^{\text{TX}} \leq P_{i,j}^{\text{TX},\max}$, $P_{S,j}^{\text{TX},\min} \leq P_{S,j}^{\text{TX}} \leq P_{S,j}^{\text{TX},\max}$, which jointly constrain the feasible processing and transmission rates, thereby implying both lower and upper bounds on the resulting service times.

IV. PROBLEM FORMULATION

Our goal is to jointly optimize the cut-layer decisions $\{L_{c,i}\}$ and the control variables $u_{i,j} = \{f_{i,j}, P_{i,j}^{\text{TX}}, f_{S,j}, P_{S,j}^{\text{TX}}\}$ to balance per-round energy consumption and training latency. Thus, we consider the optimal hybrid control problem with energy-latency tradeoff objective:

$$\mathbf{P1} : \min_{\{u_{i,j}, L_{c,i}\}} E_r(u_{i,j}, s_{i,j}, L_{c,i}) + \lambda T_r(u_{i,j}, s_{i,j}, L_{c,i}) \quad (10a)$$

$$\text{s.t. } f_i^{\min} \leq f_{i,j} \leq f_i^{\max}, \quad (10b)$$

$$f_S^{\min} \leq f_{S,j} \leq f_S^{\max}, \quad (10c)$$

$$P_{i,j}^{\text{TX},\min} \leq P_{i,j}^{\text{TX}} \leq P_{i,j}^{\text{TX},\max}, \quad (10d)$$

$$P_{S,j}^{\text{TX},\min} \leq P_{S,j}^{\text{TX}} \leq P_{S,j}^{\text{TX},\max}. \quad (10e)$$

where $\lambda > 0$ controls the tradeoff between energy costs and training latency.

A. Problem Transformation

By lifting $\{s_{i,j}\}$ as high-level decision variables, we can rewrite $T_r(z_{i,j}, u_{i,j}, s_{i,j}, L_{c,i})$ as $\hat{T}(\{s_{i,j}\})$. Specifically, the per-round latency is given by

$$\begin{aligned} T_r(u_{i,j}, s_{i,j}, L_{c,i}) \\ = \max_{i \in \mathcal{N}} \left\{ s_{i,1}(f_{i,j}, L_{c,i}) + s_{i,2}(P_{i,j}^{\text{TX}}, L_{c,i}) \right\} + \sum_{i \in \mathcal{N}} s_{i,3}(f_{S,j}, \end{aligned}$$

$$L_{c,i}) + \max_{i \in \mathcal{N}} \left\{ s_{i,4}(P_{S,j}^{\text{TX}}, L_{c,i}) + s_{i,5}(f_{i,j}, L_{c,i}) \right\} \\ \triangleq \widehat{T}(\{s_{i,j}(u_{i,j}, L_{c,i})\}), \quad (11)$$

where each $s_{i,j}$ is determined by $(u_{i,j}, L_{c,i})$ via the stage models. To handle the $\max(\cdot)$ terms, we introduce epigraph variables T_1 and T_2 such that

$$s_{i,1} + s_{i,2} \leq T_1, \quad \forall i \in \mathcal{N}, \quad (12)$$

$$s_{i,4} + s_{i,5} \leq T_2, \quad \forall i \in \mathcal{N}. \quad (13)$$

Then, minimizing $\lambda \widehat{T}(\{s_{i,j}\})$ is equivalent to minimizing $\lambda(T_1 + T_2) + \lambda \sum_{i \in \mathcal{N}} s_{i,3}$ subject to Eq. (12) and Eq. (13), because at optimality $T_1 = \max_i(s_{i,1} + s_{i,2})$ and $T_2 = \max_i(s_{i,4} + s_{i,5})$.

Minimizing the energy cost $\psi_{i,j}(u_{i,j}, s_{i,j}, L_{c,i})$ of client i at stage j for a given duration $s_{i,j}$ constitutes a classical optimal control problem with state specified at a fixed terminal time [35], [36]. Let the optimal control input be denoted as $\{\mu_{i,j}^*(s_{i,j}, L_{c,i}^*)\}$ and the optimal stage cost can be denoted as

$$\begin{aligned} \Psi_{i,j}(s_{i,j}) &= \min_{u_{i,j}(t), L_{c,i}} E_{i,j}(u_{i,j}, s_{i,j}, L_{c,i}) \\ &= \min_{\ell \in \{1, 2, \dots, L_{\max}\}} \min_{u_{i,j}(t)} E_{i,j}(u_{i,j}, s_{i,j}, L_{c,i} = \ell) \\ &= \min_{\ell \in \{1, 2, \dots, L_{\max}\}} \min_{u_{i,j}(t)} E_{i,j}^{(\ell)}(u_{i,j}, s_{i,j}) \\ &= \min_{\ell \in \{1, 2, \dots, L_{\max}\}} \widehat{\Psi}_{i,j}^{(\ell)}(s_{i,j}) \end{aligned} \quad (14)$$

where ℓ denotes each candidate cut layer and $L_{\max} \leq L$ denotes the maximum admissible cut layer. Thus, the optimal control problem is denoted by

$$\begin{aligned} \mathbf{P2}: \quad & \min_{\substack{\{s_{i,j}, y_{i,\ell}\}, \\ T_1, T_2}} \sum_{i=1}^N \sum_{j=1}^5 \sum_{\ell=1}^{L_{\max}} y_{i,\ell} \widehat{\Psi}_{i,j}^{(\ell)}(s_{i,j}) + \lambda(T_1 + T_2) \\ & + \lambda \sum_{i \in \mathcal{N}} s_{i,3} \end{aligned} \quad (15a)$$

$$\text{s.t. } s_{i,1} + s_{i,2} \leq T_1, \quad \forall i \in \mathcal{N}, \quad (15b)$$

$$s_{i,4} + s_{i,5} \leq T_2, \quad \forall i \in \mathcal{N}, \quad (15c)$$

$$\sum_{\ell=1}^{L_{\max}} S_{i,j}^{(\ell), \min} y_{i,\ell} \leq s_{i,j} \leq \sum_{\ell=1}^{L_{\max}} S_{i,j}^{(\ell), \max} y_{i,\ell}, \quad \forall i, j, \quad (15d)$$

$$\sum_{\ell=1}^{L_{\max}} y_{i,\ell} = 1, \quad y_{i,\ell} \in \{0, 1\}, \quad \forall i. \quad (15e)$$

where $y_{i,\ell}$ is a binary cut-layer selection indicator for client i , with $y_{i,\ell} = 1$ indicating that client i chooses cut layer ℓ , and $y_{i,\ell} = 0$ otherwise. The service-time bounds in (15d) couple this discrete choice with the stage durations $\{s_{i,j}\}$ by restricting $s_{i,j}$ to the range corresponding to the selected cut layer, and the constraint in (15e) enforces that each client selects exactly one cut layer. Moreover, $S_{i,j}^{(\ell), \min}$ and $S_{i,j}^{(\ell), \max}$ are denoted by

$$S_{i,j}^{(\ell), \min} =$$

$$\begin{cases} \sum_{l=1}^{\ell} \rho_l / (n_i f_i^{\max}), & j = 1, i \in \mathcal{N}, \\ d_i^U(\ell) / \left(B^U \log_2 \left(1 + \frac{P_i^{\text{TX}, \max} h_i^2}{N_0} \right) \right), & j = 2, i \in \mathcal{N}, \\ \sum_{l=\ell+1}^L (\rho_l + \varpi_l) / (n_s f_s^{\max}), & j = 3, i \in \mathcal{N}, \\ d_i^D(\ell) / \left(B^D \log_2 \left(1 + \frac{P_s^{\text{TX}, \max} h_s^2}{N_0} \right) \right), & j = 4, i \in \mathcal{N}, \\ \sum_{l=1}^{\ell} \varpi_l / (n_i f_i^{\max}), & j = 5, i \in \mathcal{N}, \end{cases} \quad (16)$$

$$\begin{aligned} S_{i,j}^{(\ell), \max} = & \begin{cases} \sum_{l=1}^{\ell} \rho_l / (n_i f_i^{\min}), & j = 1, i \in \mathcal{N}, \\ d_i^U(\ell) / \left(B^U \log_2 \left(1 + \frac{P_i^{\text{TX}, \min} h_i^2}{N_0} \right) \right), & j = 2, i \in \mathcal{N}, \\ \sum_{l=\ell+1}^L (\rho_l + \varpi_l) / (n_s f_s^{\min}), & j = 3, i \in \mathcal{N}, \\ d_i^D(\ell) / \left(B^D \log_2 \left(1 + \frac{P_s^{\text{TX}, \min} h_s^2}{N_0} \right) \right), & j = 4, i \in \mathcal{N}, \\ \sum_{l=1}^{\ell} \varpi_l / (n_i f_i^{\min}), & j = 5, i \in \mathcal{N}, \end{cases} \end{aligned} \quad (17)$$

We note that this paper does not explicitly optimize test accuracy. In our prior work [13]–[15], we showed that shallower split points generally lead to faster convergence, whereas overly deep split points may degrade learning performance. Motivated by this observation, we restrict the cut-layer design space to

$$\mathcal{L}_{\max} \triangleq \{1, 2, \dots, L_{\max}\}.$$

Accordingly, the subsequent optimization focuses on the system-level energy–latency tradeoff over the restricted cut-layer set.

To analyze the energy-latency tradeoff, we consider the Pareto efficiency associated with Problem **P2**. For any feasible decision vector ξ , define

$$E(\xi) \triangleq \sum_{i=1}^N \sum_{j=1}^5 \Psi_{i,j}(s_{i,j}), \quad (18)$$

and

$$T(\xi) \triangleq (T_1 + T_2) + \sum_{i \in \mathcal{N}} s_{i,3}, \quad (19)$$

where $\{s_{i,j}\}$, T_1 , and T_2 are components of ξ .

A feasible solution ξ^* is Pareto efficient with respect to (E, T) if there exists no feasible $\hat{\xi}$ such that $E(\hat{\xi}) \leq E(\xi^*)$ and $T(\hat{\xi}) \leq T(\xi^*)$, with at least one strict inequality. Consider the weighted-sum scalarization

$$\min_{\xi \in \mathcal{X}} F_{\lambda}(\xi) \triangleq E(\xi) + \lambda T(\xi), \quad \lambda > 0, \quad (20)$$

where $\mathcal{X}$ is the feasible set of **P2**. For any fixed $\lambda > 0$, any global minimizer of (20) is Pareto efficient. However, varying λ does not necessarily recover all Pareto-efficient solutions, because the attainable objective set $\{(E(\xi), T(\xi)) : \xi \in \mathcal{X}\}$ is generally non-convex in our setting due to the binary variables $y_{i,\ell} \in \{0, 1\}$ and the generally non-convex stage-energy term $\Psi_{i,j}(s_{i,j}) = \min_{\ell} \widehat{\Psi}_{i,j}^{(\ell)}(s_{i,j})$. Hence, (20) may recover only supported Pareto-efficient points.

V. SPECIAL CASE: MODEL-SPLITTING OPTIMIZATION

In this section, we consider model-splitting optimization for SFL, which is an important special case of **P1**. While the solution space of the integer problem is exponential, we introduce auxiliary variables to divide the joint problem into client subproblems to solve it exactly in polynomial time.

A. Model-splitting Problem Formulation

We first study a *model-splitting* optimization of the original energy-latency problem in Eq. (10). Specifically, we fix the per-stage resource controls at prescribed feasible constants, e.g., $P_{i,j}^{\text{TX}} = \hat{P}_{i,j}^{\text{TX}}$, $P_{S,j}^{\text{TX}} = \hat{P}_{S,j}^{\text{TX}}$, $f_{i,j} = \hat{f}_{i,j}$, $f_{S,j} = \hat{f}_{S,j}$, where $\hat{P}_{i,j}^{\text{TX}}$, $\hat{P}_{S,j}^{\text{TX}}$, $\hat{f}_{i,j}$, and $\hat{f}_{S,j}$ denote prescribed feasible constants for the client transmit power, server transmit power, client GPU frequency, and server GPU frequency, respectively. In this case, for each candidate cut layer $\ell \in \{1, \dots, L_{\max}\}$, the corresponding stage time and energy become deterministic constants, denoted by

$$S_{i,j}^{[\ell]} = s_{i,j}(\hat{f}_{i,j}, \hat{P}_{i,j}^{\text{TX}}, \hat{f}_{S,j}, \hat{P}_{S,j}^{\text{TX}}, \ell), \quad (21)$$

$$E_{i,j}^{[\ell]} = E_{i,j}(\hat{f}_{i,j}, \hat{P}_{i,j}^{\text{TX}}, \hat{f}_{S,j}, \hat{P}_{S,j}^{\text{TX}}, \ell). \quad (22)$$

Furthermore, we introduce binary decision variables $y_{i,\ell} \in \{0, 1\}$ indicating whether client i selects cut layer ℓ , i.e.,

$$\sum_{\ell=1}^{L_{\max}} y_{i,\ell} = 1, \quad y_{i,\ell} \in \{0, 1\}, \quad \forall i \in \mathcal{N}. \quad (23)$$

Then the stage time of client i can be rewritten as affine selections:

$$s_{i,j} = \sum_{\ell=1}^{L_{\max}} S_{i,j}^{[\ell]} y_{i,\ell}, \quad \forall i, j, \quad (24)$$

Substituting (24) into (6)–(10), we obtain

$$\begin{aligned} \mathbf{P3}: \min_{\{y_{i,\ell}\}} & \sum_{i \in \mathcal{N}} \sum_{\ell=1}^{L_{\max}} E_{i,j}^{[\ell]} y_{i,\ell} + \lambda \left(\max_{i \in \mathcal{N}} \sum_{\ell=1}^{L_{\max}} (S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]}) y_{i,\ell} \right. \\ & \left. + \sum_{i \in \mathcal{N}} \sum_{\ell=1}^{L_{\max}} S_{i,3}^{[\ell]} y_{i,\ell} + \max_{i \in \mathcal{N}} \sum_{\ell=1}^{L_{\max}} (S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]}) y_{i,\ell} \right) \end{aligned} \quad (25a)$$

$$\text{s.t.} \quad \sum_{\ell=1}^{L_{\max}} y_{i,\ell} = 1, \quad y_{i,\ell} \in \{0, 1\}, \quad \forall i \in \mathcal{N}. \quad (25b)$$

where $E_i^{[\ell]} \triangleq \sum_{j=1}^5 E_{i,j}^{[\ell]} (\forall i, \ell)$. Problem **P3** in (25a) is a 0–1 discrete optimization that searches for the best cut layer across clients with exponentially cut-layer configurations in general. Despite its complexity, we will develop a polynomial-time algorithm to solve it exactly in the subsequent development.

B. Solution Approach

We introduce two variables T_1 and T_2 satisfying:

$$\sum_{\ell=1}^{L_{\max}} (S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]}) y_{i,\ell} \leq T_1, \quad \forall i \in \mathcal{N}, \quad (26)$$

$$\sum_{\ell=1}^{L_{\max}} (S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]}) y_{i,\ell} \leq T_2, \quad \forall i \in \mathcal{N}. \quad (27)$$

Then, Problem **P3** can be equivalently transformed into

$$\mathbf{P3'}: \min_{\{y_{i,\ell}\}, T_1, T_2} \sum_{i \in \mathcal{N}} \sum_{\ell=1}^{L_{\max}} (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}) y_{i,\ell} + \lambda(T_1 + T_2) \quad (28a)$$

$$\text{s.t.} \quad \sum_{\ell=1}^{L_{\max}} y_{i,\ell} = 1, \quad y_{i,\ell} \in \{0, 1\}, \quad \forall i \in \mathcal{N}, \quad (28b)$$

$$\sum_{\ell=1}^{L_{\max}} (S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]}) y_{i,\ell} \leq T_1, \quad \forall i \in \mathcal{N}, \quad (28c)$$

$$\sum_{\ell=1}^{L_{\max}} (S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]}) y_{i,\ell} \leq T_2, \quad \forall i \in \mathcal{N}. \quad (28d)$$

To obtain the optimal solution, we enumerate the candidate thresholds according to

$$T_1 \in \{S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]}\}, \quad (29)$$

$$T_2 \in \{S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]}\}, \quad (30)$$

which are finite sets of cardinality at most $N(L-1)$. For any fixed (T_1, T_2) , client i selects the cut layer with minimum cost:

$$\ell_i^*(T_1, T_2) = \arg \min_{\ell \in \mathcal{F}_i(T_1, T_2)} (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}), \quad (31)$$

where the feasible cut-layer set $\mathcal{F}_i(T_1, T_2)$ is defined as

$$\mathcal{F}_i(T_1, T_2) \triangleq \{\ell : S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]} \leq T_1, S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]} \leq T_2\}. \quad (32)$$

The resulting objective value is

$$J(T_1, T_2) = \underbrace{\sum_{i \in \mathcal{N}} \min_{\ell \in \mathcal{F}_i(T_1, T_2)} (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]})}_{G(T_1, T_2): \text{Non-increasing}} + \underbrace{\lambda(T_1 + T_2)}_{H(T_1, T_2): \text{Strictly increasing}}. \quad (33)$$

Finally, the globally optimal thresholds are obtained by

$$(T_1^*, T_2^*) = \arg \min_{T_1, T_2} J(T_1, T_2), \quad (34)$$

which yields a globally optimal cut-layer decision $\{y_{i,\ell}^*\}$ for (25a).

To solve Problem **P3**, **Algorithm 1** first constructs the candidate threshold sets

$$\mathcal{T}_1 \triangleq \{S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]} \mid i \in \mathcal{N}, \ell \in \{1, \dots, L_{\max}\}\},$$

$$\mathcal{T}_2 \triangleq \{S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]} \mid i \in \mathcal{N}, \ell \in \{1, \dots, L_{\max}\}\}.$$

The two sets are sorted in ascending order and enumerated as candidate threshold values. For each fixed $T_2 \in \mathcal{T}_2$, the algorithm sequentially sweeps $T_1 \in \mathcal{T}_1$, updates the best feasible cut-layer choice for each client, and applies a bound-based pruning rule to terminate the T_1 scan early whenever no better objective value can be attained. By enumerating all candidate threshold pairs together with this pruning rule, **Algorithm 1** solves Problem **P3** to global optimality.

Algorithm 1: Optimal Model-splitting Optimization via Plane Sweep and Pruning

Input: $\{S_{i,j}^{[\ell]}\}, \{E_i^{[\ell]}\}, \lambda, V_i^{[\ell]} \triangleq E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}$.
Output: $J^*, \{\ell_i^*\}$.
 $\mathcal{T}_1 \leftarrow \text{sort}(\{S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]}\}), \mathcal{T}_2 \leftarrow \text{sort}(\{S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]}\});$
 $G_{\min} \leftarrow \sum_{i \in \mathcal{N}} \min_{\ell \in \{1, \dots, L_{\max}\}} (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]});$
 $\mathcal{B}[t] \leftarrow \{(i, \ell) \mid S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]} = t\}; J^* \leftarrow \infty;$
foreach $T_2 \in \mathcal{T}_2$ **do**
 // outer loop: fix T_2
 $c_i \leftarrow \infty, \ell_i \leftarrow -1 (\forall i); G \leftarrow 0; \text{count} \leftarrow 0;$
 for $k \leftarrow 1$ **to** $|\mathcal{T}_1|$ **do**
 // inner loop: sweep T_1
 $T_1 \leftarrow \mathcal{T}_1[k];$
 foreach $(i, \ell) \in \mathcal{B}[T_1]$ **do**
 // process pairs introduced by the current T_1
 if $S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]} \leq T_2$ **and** $(E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}) < c_i$ **then**
 // passes T_2 check and reduces client- i cost
 if $c_i = \infty$ **then**
 $\text{count} \leftarrow \text{count} + 1;$
 $G \leftarrow G + (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]});$
 else
 // Replace the previous best cut of client i
 $G \leftarrow G - c_i + (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]});$
 end
 $c_i \leftarrow (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}); \ell_i \leftarrow \ell;$
 end
 end
 if $\text{count} = |\mathcal{N}|$ **then**
 $J \leftarrow G + \lambda(T_1 + T_2);$
 if $J < J^*$ **then**
 $J^* \leftarrow J; \{\ell_i^*\} \leftarrow \{\ell_i\}$
 end
 // Prune the remaining T_1 candidates for this fixed T_2 (**Theorem 1**)
 if $k < |\mathcal{T}_1|$ **and** $\lambda(\mathcal{T}_1[k+1] - \mathcal{T}_1[k]) > G - G_{\min}$ **then**
 break
 end
 end
 end
end
return $J^*, \{\ell_i^*\};$

Theorem 1 (Optimality of **Algorithm 1**). *Algorithm 1 obtains the globally optimal solution to Problem P3.*

Proof. Fix any $T_2 \in \mathcal{T}_2$. Let $\{\tau_{1,k}\}_{k=1}^{|\mathcal{T}_1|}$ denote the elements of $\mathcal{T}_1$ sorted in ascending order, i.e., $\tau_{1,1} \leq \tau_{1,2} \leq \dots \leq \tau_{1,|\mathcal{T}_1|}$. Recall $J(T_1, T_2) = G(T_1, T_2) + \lambda(T_1 + T_2)$, where $G(\cdot, T_2)$ is non-increasing in T_1 and is lower bounded by $G_{\min} \triangleq$

$$\sum_{i \in \mathcal{N}} \min_{\ell \in \{1, \dots, L_{\max}\}} (E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}).$$

At step k , for any future candidate $\tau_{1,m} > \tau_{1,k}$, we have

$$\begin{aligned} J(\tau_{1,m}, T_2) - J(\tau_{1,k}, T_2) &= (G(\tau_{1,m}, T_2) - G(\tau_{1,k}, T_2)) + \lambda(\tau_{1,m} - \tau_{1,k}) \\ &\geq (G_{\min} - G(\tau_{1,k}, T_2)) + \lambda(\tau_{1,m} - \tau_{1,k}), \end{aligned}$$

where the inequality uses $G(\tau_{1,m}, T_2) \geq G_{\min}$. Since $\tau_{1,m} - \tau_{1,k} \geq \tau_{1,k+1} - \tau_{1,k}$, it follows that

$$J(\tau_{1,m}, T_2) - J(\tau_{1,k}, T_2) \geq -\Delta G_{\max} + \lambda(\tau_{1,k+1} - \tau_{1,k}),$$

with $\Delta G_{\max} \triangleq G(\tau_{1,k}, T_2) - G_{\min}$. Thus, if $\lambda(\tau_{1,k+1} - \tau_{1,k}) > \Delta G_{\max}$, then $J(\tau_{1,m}, T_2) > J(\tau_{1,k}, T_2)$ for all $m > k$, so no better solution exists beyond $\tau_{1,k}$ and the scan over T_1 can be safely terminated for this fixed T_2 . Furthermore, enumerating all $T_2 \in \mathcal{T}_2$ ensures that the global minimizer (T_1^*, T_2^*) is not missed, hence the algorithm returns a globally optimal $\{y_{i,\ell}^*\}$. $\square$

Complexity. **Algorithm 1** first constructs the candidate sets $\mathcal{T}_1 = \{S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]}\}$ and $\mathcal{T}_2 = \{S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]}\}$ and sorts them, which costs $O(NL \log(NL))$ since $|\mathcal{T}_1|, |\mathcal{T}_2| \leq N(L-1)$. For each fixed $T_2 \in \mathcal{T}_2$, **Algorithm 1** scans $T_1 \in \mathcal{T}_1$ in ascending order. Because T_1 is enumerated over the sorted distinct values in $\mathcal{T}_1$, the uplink-feasible set $\{(i, \ell) \mid S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]} \leq T_1\}$ expands only when T_1 advances to the next candidate; hence, at a given T_1 it suffices to process only the newly activated pairs (i, ℓ) satisfying $S_{i,1}^{[\ell]} + S_{i,2}^{[\ell]} = T_1$. For each such pair, the algorithm performs (i) one comparison to test downlink feasibility $S_{i,4}^{[\ell]} + S_{i,5}^{[\ell]} \leq T_2$, (ii) one evaluation of $E_i^{[\ell]} + \lambda S_{i,3}^{[\ell]}$, and (iii) a constant-time comparison and possible update of the stored best value c_i together with the running sum. All these operations take $O(1)$ time per processed pair. Thus, over a complete scan of T_1 for a fixed T_2 , each (i, ℓ) is processed at most once, and the total update cost per T_2 is $O(NL)$. Consequently, the overall running time of **Algorithm 1** is

$$\begin{aligned} &O(|\mathcal{T}_2| \cdot NL) + O(NL \log(NL)) \\ &= O(N^2 L^2 + NL \log(NL)) \\ &= O(N^2 L^2). \end{aligned} \tag{35}$$

VI. JOINT OPTIMIZATION OF MODEL SPLITTING AND RESOURCE ALLOCATION

We now present a polynomial-time approximation method for the general joint optimization of model splitting and resource allocation. Specifically, we first reformulate Problem **P2** exactly as a two-dimensional master problem with respect to the synchronization variables (T_1, T_2) . Since the resulting master problem is generally nonconvex, we then approximate it via a uniform grid search over the (T_1, T_2) domain. This leads to a polynomial-time approximation algorithm with an explicit performance guarantee.

A. Two-Dimensional Bottleneck Projection

Recall that the latency term in Problem **P2** is characterized by the two epigraph variables T_1 and T_2 , where

$$s_{i,1} + s_{i,2} \leq T_1, \quad \forall i \in \mathcal{N}, \tag{36}$$

$$s_{i,4} + s_{i,5} \leq T_2, \quad \forall i \in \mathcal{N}. \quad (37)$$

Once (T_1, T_2) is fixed, the optimization variables across clients become decoupled. This key observation enables us to project the original mixed-integer problem onto the two synchronization variables. Specifically, for each client i , stage j , and cut layer $\ell \in \{1, 2, \dots, L_{\max}\}$, let $\widehat{\Psi}_{i,j}^{(\ell)}(s_{i,j})$ denote the minimum stage energy when the service time is $s_{i,j}$ and the cut layer is fixed at ℓ , i.e.,

$$\widehat{\Psi}_{i,j}^{(\ell)}(s_{i,j}) = \min_{u_{i,j}(t)} E_{i,j}(u_{i,j}, s_{i,j}, L_{c,i}=\ell). \quad (38)$$

Moreover, the feasible service-time is given by

$$S_{i,j}^{(\ell),\min} \leq s_{i,j} \leq S_{i,j}^{(\ell),\max}. \quad (39)$$

For fixed (T_1, T_2) , the per-client optimization decomposes into three independent subproblems. First, the server-side FP/BP stage contributes additively to the objective and is not directly constrained by T_1 or T_2 . Accordingly, we define

$$\mathbf{P4}' : U_{i,3}^{(\ell)} \triangleq \min_{s_{i,3}} \widehat{\Psi}_{i,3}^{(\ell)}(s_{i,3}) + \lambda s_{i,3} \quad (40a)$$

$$\text{s.t. } S_{i,3}^{(\ell),\min} \leq s_{i,3} \leq S_{i,3}^{(\ell),\max}. \quad (40b)$$

Second, the first synchronization variable T_1 constrains the sum of client-side FP time and uplink transmission time, which is given by

$$\mathbf{P4}'' : U_{i,12}^{(\ell)}(T_1) \triangleq \min_{s_{i,1}, s_{i,2}} \widehat{\Psi}_{i,1}^{(\ell)}(s_{i,1}) + \widehat{\Psi}_{i,2}^{(\ell)}(s_{i,2}) \quad (41a)$$

$$\text{s.t. } s_{i,1} + s_{i,2} \leq T_1, \quad (41b)$$

$$S_{i,1}^{(\ell),\min} \leq s_{i,1} \leq S_{i,1}^{(\ell),\max}, \quad (41c)$$

$$S_{i,2}^{(\ell),\min} \leq s_{i,2} \leq S_{i,2}^{(\ell),\max}. \quad (41d)$$

If $T_1 < S_{i,1}^{(\ell),\min} + S_{i,2}^{(\ell),\min}$, then cut layer ℓ is infeasible for client i under T_1 , and we set $U_{i,12}^{(\ell)}(T_1) = +\infty$.

Third, considering T_2 constrains the sum of the downlink transmission time and client-side BP time, we define

$$\mathbf{P4}''' : U_{i,45}^{(\ell)}(T_2) \triangleq \min_{s_{i,4}, s_{i,5}} \widehat{\Psi}_{i,4}^{(\ell)}(s_{i,4}) + \widehat{\Psi}_{i,5}^{(\ell)}(s_{i,5}) \quad (42a)$$

$$\text{s.t. } s_{i,4} + s_{i,5} \leq T_2, \quad (42b)$$

$$S_{i,4}^{(\ell),\min} \leq s_{i,4} \leq S_{i,4}^{(\ell),\max}, \quad (42c)$$

$$S_{i,5}^{(\ell),\min} \leq s_{i,5} \leq S_{i,5}^{(\ell),\max}. \quad (42d)$$

If $T_2 < S_{i,4}^{(\ell),\min} + S_{i,5}^{(\ell),\min}$, then cut layer ℓ is infeasible for client i under T_2 , and we set $U_{i,45}^{(\ell)}(T_2) = +\infty$.

For fixed (T_1, T_2) , client i evaluates all candidate cut layers and chooses the least-cost feasible one. This yields the client value function

$$V_i(T_1, T_2) \triangleq \min_{\ell \in \{1, 2, \dots, L_{\max}\}} \left(U_{i,12}^{(\ell)}(T_1) + U_{i,45}^{(\ell)}(T_2) + U_{i,3}^{(\ell)} \right). \quad (43)$$

Hence, the original mixed-integer Problem **P2** is exactly reduced to the following two-dimensional master problem:

$$\mathbf{P5} : \min_{T_1, T_2} J(T_1, T_2) \triangleq \lambda(T_1 + T_2) + \sum_{i \in \mathcal{N}} V_i(T_1, T_2). \quad (44)$$

where $V_i(T_1, T_2)$ is defined in (43). The feasibility conditions are already incorporated into the definitions of $U_{i,3}^{(\ell)}$, $U_{i,12}^{(\ell)}(T_1)$, and $U_{i,45}^{(\ell)}(T_2)$, with infeasible choices assigned value $+\infty$.

Theorem 2 (Problem Equivalence). *Problem **P2** is equivalent to the master problem in (44). In particular, solving Problem **P5** yields an optimal solution to Problem **P2**.*

Proof. Fix any feasible pair (T_1, T_2) . Under this pair, the only coupling among clients is removed, and the optimization variables of different clients become separable. For each client i and each cut layer ℓ , the optimal contribution to the objective decomposes into the three blocks $U_{i,12}^{(\ell)}(T_1)$, $U_{i,45}^{(\ell)}(T_2)$ and $U_{i,3}^{(\ell)}$. Minimizing over $\ell \in \{1, 2, \dots, L_{\max}\}$ gives the client value function $V_i(T_1, T_2)$ in (43). Summing the optimal client values over all clients and adding the synchronization penalty $\lambda(T_1 + T_2)$ yields (44). Thus, minimizing Problem **P5** over (T_1, T_2) is equivalent to solving Problem **P2**. $\square$

B. Grid Approximation of the Master Problem

The reformulation in (44) is equivalent to the original problem. However, the master objective $J(T_1, T_2)$ is generally nonconvex because each client value function $V_i(T_1, T_2)$ contains a minimization over discrete cut layers. Since the master problem is only two-dimensional, we approximate it via a uniform grid search over the bounded rectangle

$$T_1 \in [\underline{T}_1, \bar{T}_1], \quad T_2 \in [\underline{T}_2, \bar{T}_2], \quad (45)$$

where the search box is chosen to contain at least one global optimizer of (44). For a grid step size $\Delta > 0$, we evaluate $J(T_1, T_2)$ at all grid points in (45) and return the best value, denoted by J_{grid} . The following theorem provides an additive approximation guarantee.

Theorem 3 (Relative Error Bound). *Assume that the search region $[\underline{T}_1, \bar{T}_1] \times [\underline{T}_2, \bar{T}_2]$ contains at least one global optimizer (T_1^*, T_2^*) of (44), and let $J^* \triangleq J(T_1^*, T_2^*)$. Suppose that $J^* \geq \lambda(\underline{T}_1 + \underline{T}_2)$, where $\underline{T}_1 + \underline{T}_2 > 0$. If a uniform grid with step size Δ is used in both dimensions, then*

$$\frac{J_{\text{grid}} - J^*}{J^*} \leq \frac{2\Delta}{\underline{T}_1 + \underline{T}_2}.$$

Consequently, in order to guarantee $J_{\text{grid}} \leq (1 + \epsilon)J^$, it suffices to choose $\Delta \leq \frac{\epsilon}{2}(\underline{T}_1 + \underline{T}_2)$.*

Proof. Since the grid spacing is Δ , there exists a grid point $(\tilde{T}_1, \tilde{T}_2)$ such that

$$T_1^* \leq \tilde{T}_1 \leq T_1^* + \Delta,$$

$$T_2^* \leq \tilde{T}_2 \leq T_2^* + \Delta.$$

For each client $i \in \mathcal{N}$, increasing T_1 or T_2 relaxes the corresponding synchronization constraints and thus cannot increase the value function $V_i(T_1, T_2)$. Hence,

$$V_i(\tilde{T}_1, \tilde{T}_2) \leq V_i(T_1^*, T_2^*), \quad \forall i \in \mathcal{N}.$$

It follows that

$$\begin{aligned} J(\tilde{T}_1, \tilde{T}_2) &= \lambda(\tilde{T}_1 + \tilde{T}_2) + \sum_{i \in \mathcal{N}} V_i(\tilde{T}_1, \tilde{T}_2) \\ &\leq \lambda(T_1^* + \Delta + T_2^* + \Delta) + \sum_{i \in \mathcal{N}} V_i(T_1^*, T_2^*) \\ &= J^* + 2\lambda\Delta. \end{aligned}$$

Since the grid-search algorithm returns the best grid point, we further have

$$J_{\text{grid}} \leq J(\tilde{T}_1, \tilde{T}_2).$$

Then, we have $J_{\text{grid}} - J^* \leq 2\lambda\Delta$. Dividing both sides by J^* and using $J^* \geq \lambda(\underline{T}_1 + \underline{T}_2)$, we obtain

$$\frac{J_{\text{grid}} - J^*}{J^*} \leq \frac{2\lambda\Delta}{J^*} \leq \frac{2\Delta}{\underline{T}_1 + \underline{T}_2}.$$

The desired $(1 + \epsilon)$ -approximation guarantee is obtained by requiring $2\Delta/(\underline{T}_1 + \underline{T}_2) \leq \epsilon$, which completes the proof. $\square$

Theorem 4 (Complexity of Grid Approximation). *Let $\eta > 0$ denote the numerical accuracy to solve the inner subproblems. Then, the proposed grid-search algorithm has total complexity $O(G_1 G_2 N L \log(1/\eta))$. Under bounded search ranges, choosing Δ according to **Theorem 3** yields the total complexity $O((NL/\epsilon^2) \log(1/\eta))$. Hence, **Algorithm 2** runs in polynomial time.*

Proof. 1) *Complexity of the inner subproblems.* Fix any grid point (T_1, T_2) , client i , and candidate cut layer ℓ . For the corresponding client i pair (i, ℓ) with cut layer ℓ , the algorithm evaluates the three inner quantities $U_{i,3}^{(\ell)}$, $U_{i,12}^{(\ell)}(T_1)$, $U_{i,45}^{(\ell)}(T_2)$. We first show that these inner subproblems are convex. For the computation stages $j \in \{1, 3, 5\}$, under a fixed cut layer ℓ , the stage-energy functions are given by

$$\hat{\Psi}_{i,j}^{(\ell)}(s_{i,j}) = a_{i,j}^{(\ell)} / s_{i,j}^2, \quad j \in \{1, 3, 5\},$$

where

$$a_{i,j}^{(\ell)} = \begin{cases} G_i \left(\sum_{k=1}^{\ell} \rho_k \right)^3 / n_i^3, & j = 1, \\ G_S \left(\sum_{k=\ell+1}^L (\rho_k + \varpi_k) \right)^3 / n_S^3, & j = 3, \\ G_i \left(\sum_{k=1}^{\ell} \varpi_k \right)^3 / n_i^3, & j = 5. \end{cases}$$

Since

$$\frac{d^2}{ds_{i,j}^2} \left(\frac{a_{i,j}^{(\ell)}}{s_{i,j}^2} \right) = \frac{6a_{i,j}^{(\ell)}}{s_{i,j}^4} > 0, \quad \forall s_{i,j} > 0,$$

it follows that $\hat{\Psi}_{i,j}^{(\ell)}(s_{i,j})$ is convex on $s_{i,j} > 0$.

For the communication stages $j \in \{2, 4\}$, under a fixed cut layer ℓ , the stage-energy functions can be written as

$$\hat{\Psi}_{i,j}^{(\ell)}(s_{i,j}) = b_{i,j}^{(\ell)} s_{i,j} \left(2^{c_{i,j}^{(\ell)}/s_{i,j}} - 1 \right), \quad j \in \{2, 4\},$$

Algorithm 2: Grid Approximation for the Two-Dimensional Master Problem

Input: $\mathcal{N}, \mathcal{L}, [\underline{T}_1, \bar{T}_1], [\underline{T}_2, \bar{T}_2], \Delta, \lambda$.

Output: $J^*, (T_1^*, T_2^*), \{\ell_i^*\}$.

$\mathcal{G}_1 \leftarrow \{\underline{T}_1, \underline{T}_1 + \Delta, \underline{T}_1 + 2\Delta, \dots, \bar{T}_1\}$;

$\mathcal{G}_2 \leftarrow \{\underline{T}_2, \underline{T}_2 + \Delta, \underline{T}_2 + 2\Delta, \dots, \bar{T}_2\}$;

$J^* \leftarrow \infty$;

foreach $T_1 \in \mathcal{G}_1$ **do**

foreach $T_2 \in \mathcal{G}_2$ **do**

$J \leftarrow \lambda(T_1 + T_2)$;

 feasible $\leftarrow 1$;

 Compute $U_{i,3}^{(\ell)}$, $U_{i,12}^{(\ell)}(T_1)$, and $U_{i,45}^{(\ell)}(T_2)$ for all required (i, ℓ, T_1, T_2) combinations;

foreach $i \in \mathcal{N}$ **do**

$V_i \leftarrow \infty$, $\ell_i \leftarrow -1$;

foreach $\ell \in \mathcal{L}$ **do**

if $T_1 < S_{i,1}^{(\ell),\min} + S_{i,2}^{(\ell),\min}$ **or**

$T_2 < S_{i,4}^{(\ell),\min} + S_{i,5}^{(\ell),\min}$ **then**

continue;

end

$C_i^{(\ell)} \leftarrow U_{i,3}^{(\ell)} + U_{i,12}^{(\ell)}(T_1) + U_{i,45}^{(\ell)}(T_2)$;

if $C_i^{(\ell)} < V_i$ **then**

$V_i \leftarrow C_i^{(\ell)}$, $\ell_i \leftarrow \ell$;

end

end

if $V_i = \infty$ **then**

 feasible $\leftarrow 0$;

break;

end

$J \leftarrow J + V_i$;

end

if feasible = 1 **and** $J < J^*$ **then**

$J^* \leftarrow J$, $(T_1^*, T_2^*) \leftarrow (T_1, T_2)$, $\{\ell_i^*\} \leftarrow \{\ell_i\}$;

end

end

end

return $J^*, (T_1^*, T_2^*), \{\ell_i^*\}$;

where

$$b_{i,j}^{(\ell)} = \begin{cases} \frac{B^U N_0}{h_i^2}, & j = 2, \\ \frac{B^D N_0}{h_S^2}, & j = 4, \end{cases} \quad c_{i,j}^{(\ell)} = \begin{cases} \frac{d_i^U(\ell)}{B^U}, & j = 2, \\ \frac{d_i^D(\ell)}{B^D}, & j = 4. \end{cases}$$

Define

$$\phi_{i,j}^{(\ell)}(s_{i,j}) \triangleq b_{i,j}^{(\ell)} s_{i,j} \left(2^{c_{i,j}^{(\ell)}/s_{i,j}} - 1 \right), \quad s_{i,j} > 0.$$

Then

$$\frac{d^2}{ds_{i,j}^2} \phi_{i,j}^{(\ell)}(s_{i,j}) = \frac{b_{i,j}^{(\ell)} 2^{c_{i,j}^{(\ell)}/s_{i,j}} (c_{i,j}^{(\ell)})^2 (\ln 2)^2}{s_{i,j}^3} > 0, \quad \forall s_{i,j} > 0.$$

Hence, $\hat{\Psi}_{i,j}^{(\ell)}(s_{i,j})$ is convex for $j \in \{2, 4\}$.

Thus, $U_{i,3}^{(\ell)}$ is a one-dimensional convex minimization over

the interval $[S_{i,3}^{(\ell),\min}, S_{i,3}^{(\ell),\max}]$. Moreover, $U_{i,12}^{(\ell)}(T_1)$ and $U_{i,45}^{(\ell)}(T_2)$ are fixed-dimensional convex programs with box constraints and one affine coupling constraint. Hence, these inner subproblems can be solved efficiently. In particular, $U_{i,3}^{(\ell)}$ admits a closed-form solution after projecting the unconstrained minimizer onto its feasible interval, while $U_{i,12}^{(\ell)}(T_1)$ and $U_{i,45}^{(\ell)}(T_2)$ can be solved to accuracy η in time polynomial in $\log(1/\eta)$ by standard convex optimization methods. Thus, the total cost per client-cut-layer pair is $O(\log(1/\eta))$.

2) *Complexity of the outer grid search.* Let

$$G_1 \triangleq \left\lceil \frac{\bar{T}_1 - \underline{T}_1}{\Delta} \right\rceil + 1, \quad G_2 \triangleq \left\lceil \frac{\bar{T}_2 - \underline{T}_2}{\Delta} \right\rceil + 1$$

denote the numbers of grid points along the two dimensions. Then, the grid contains $G_1 G_2$ points in total. At each grid point, the algorithm evaluates all N clients and at most $L - 1$ candidate cut layers. Combining the complexity of the inner subproblems, the total complexity is

$$O(G_1 G_2 N (L - 1) \log(1/\eta)) = O(G_1 G_2 N L \log(1/\eta)).$$

Finally, under bounded search ranges, **Theorem 3** implies that $\Delta = O(\epsilon)$. Hence, $G_1 = O(1/\epsilon)$, $G_2 = O(1/\epsilon)$, and thus $G_1 G_2 = O(1/\epsilon^2)$. Substituting this into the above bound yields $O((NL/\epsilon^2) \log(1/\eta))$, which completes the proof. $\square$

C. Recovering the Optimal Control Inputs

Once the optimal solution of the high-level problem $\mathbf{P}$ is obtained, the optimal solution to the original hybrid control problem can be recovered by solving the lower-level control subproblems stage by stage. In particular, the high-level optimizer determines the optimal cut-layer decisions $\{L_{c,i}^*\}$ and the optimal service-time variables $\{s_{i,j}^*\}$. With these decisions solved, the corresponding optimal control input is recovered as

$$u_{i,j}^*(s_{i,j}^*) = \arg \min_{u_{i,j}} \psi_{i,j}(z_{i,j}, u_{i,j}, s_{i,j}^*, L_{c,i}^*), \quad (46)$$

subject to the associated system dynamics and boundary conditions. Thus, the optimal policy of the original hybrid control problem is obtained by combining the high-level optimal resource-allocation decisions with the recovered low-level control inputs.

VII. SIMULATIONS

A. Simulation Setup

We use the datasets MNIST [37] and CIFAR-10 [38] to evaluate the performance of the proposed method for the models ResNet-50 [39] and VGG-16 [40] under both IID and non-IID data distributions. Notably, for the non-IID setting, each client is assigned a primary class from which approximately 70% of its samples are drawn, while the remaining samples are sourced from other classes. We consider a network of $N = 20$ clients, unless specified otherwise. Each client uses a mini-batch size of 32 with a learning rate $\gamma = 0.001$. The algorithms are run on a workstation equipped with an AMD Ryzen Threadripper PRO 5975WX and NVIDIA GeForce RTX 4090.

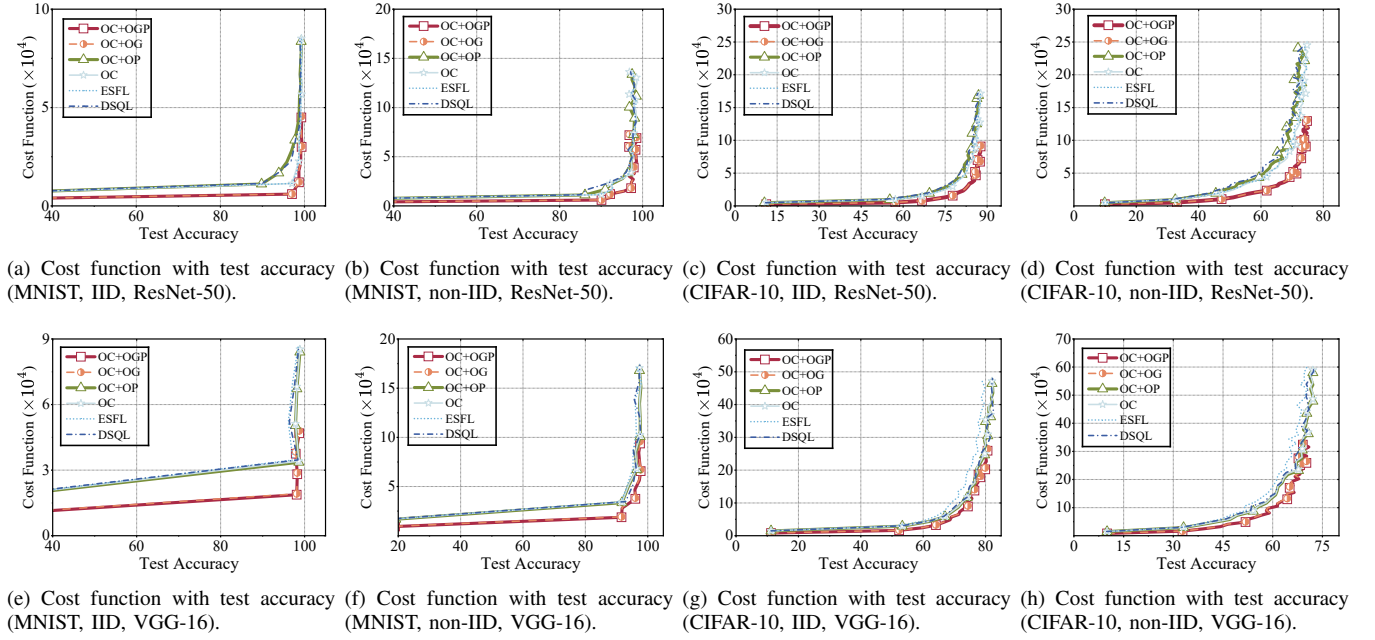
In the experiments, the tradeoff coefficient is set to $\lambda = 100$ to balance training latency and energy consumption. The client-side computation capability is heterogeneous, with n_i uniformly selected from $[0.2, 3] \times 10^4$ FLOPs/cycle, while the edge server is configured with $n_S = 3 \times 10^4$ FLOPs/cycle. The computation-energy coefficients are set to $G_i = 1 \times 10^{-26} \text{ W}/(\text{cycle/s})^3$ for clients and $G_S = 2 \times 10^{-26} \text{ W}/(\text{cycle/s})^3$ for the server. For the NVIDIA Jetson Orin Nano client platform, the computing frequency is configured within the measured graphics-clock range, namely from $f_i^{\min} = 50$ MHz to $f_i^{\max} = 625$ MHz. For the NVIDIA GeForce RTX 4090 server platform, the computing frequency is configured within the supported graphics-clock range measured on our testbed, namely from $f_S^{\min} = 210$ MHz to $f_S^{\max} = 3.135$ GHz [41], [42]. For wireless communication, the uplink and downlink bandwidths are set to $B_i^U = 100$ MHz and $B_i^D = 200$ MHz, respectively. The transmit power of each client and the edge server are bounded within $[20, 33]$ dBm. The uplink channels are modeled as independent Rayleigh fading channels, where the average channel power gain of client i is denoted by $|h_i|^2$ and generated within $[0.1, 1] \times 10^{-6}$. Similarly, the average downlink channel power gain is denoted by $|h_S|^2$ and fixed at 1×10^{-6} [34]. The noise power spectral density is set to $N_0 = 3.98 \times 10^{-21} \text{ W/Hz}$. For the reader's convenience, the detailed simulation parameters are summarized in Table II.

To demonstrate the advantages of our proposed method “Optimal Cut and Optimal GPU Frequency Scaling and Power Control” (**OC + OGP**), we compare it with five benchmarks: 1) “Optimal Cut and Optimal GPU Frequency Scaling” (**OC + OG**), which optimizes the model partition point and GPU frequency; 2) “Optimal Cut and Optimal Power Control” (**OC + OP**), which jointly optimizes the model partition point and transmit power; 3) “Optimal Cut” (**OC**), which optimizes only the model partition; 4) efficient split federated learning (**ESFL**) [17], which aims to reduce training latency by optimizing client-side workload and server-side workload; and 5) dynamic-step Q-Learning method (**DSQL**) [23], which employs a reinforcement learning method to reduce the weighted sum of training time and energy consumption. In our experiments, the total cost is defined as the cumulative cost incurred until the model reaches 70% test accuracy on CIFAR-10 under both IID and non-IID data distributions.

B. Performance Comparison

Fig. 3 evaluates the training performance by showing the cost function against test accuracy using ResNet-50 and VGG-16 models on both MNIST and CIFAR-10 datasets. The proposed **OC+OGP** method consistently achieves the lowest cost for attaining a given test accuracy, demonstrating that the proposed method remains effective across different datasets, data distributions, and model architectures. It is also observed that the cost generally increases with the test accuracy, and the increase becomes faster as the test accuracy approaches convergence.

Fig. 4 illustrates the impact of varying uplink and downlink bandwidths on the total cost function. As the uplink or downlink bandwidth increases, the overall cost gradually decreases

Fig. 3. Evaluating training performance on CIFAR-10 and MNIST using ResNet-50 and VGG-16 ($\lambda = 100$).TABLE II
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
C	20	N_0	3.98×10^{-21} W/Hz
J	5	λ	100
n_i	$[0.2, 3] \times 10^4$ FLOPs/cycle	n_s	3×10^4 FLOPs/cycle
G_i	1×10^{-26} W/(cycle/s) ³	G_s	2×10^{-26} W/(cycle/s) ³
$f_i^{\max}$	625 MHz	$f_i^{\min}$	50 MHz
$f_s^{\max}$	3.135 GHz	$f_s^{\min}$	210 MHz
B_i^U	100 MHz	B_i^D	200 MHz
$P_i^{\max}$	33 dBm	$P_i^{\min}$	20 dBm
$P_s^{\max}$	33 dBm	$P_s^{\min}$	20 dBm
$ h_i ^2$	$[0.1, 1] \times 10^{-6}$	$ h_s ^2$	1×10^{-6}

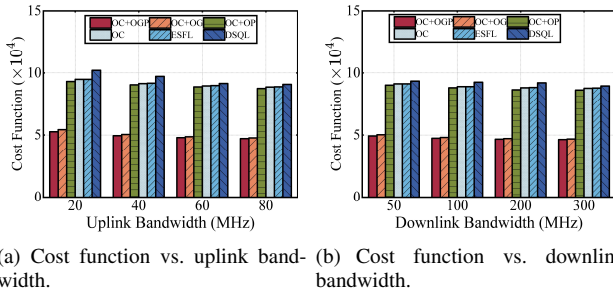


Fig. 4. The impact of uplink bandwidth and downlink bandwidth on cost function.

for all compared baselines, since larger bandwidth improves transmission efficiency and shortens the communication time. Moreover, the proposed **OC+OGP** consistently achieves the lowest cost under all evaluated bandwidth settings, indicat-

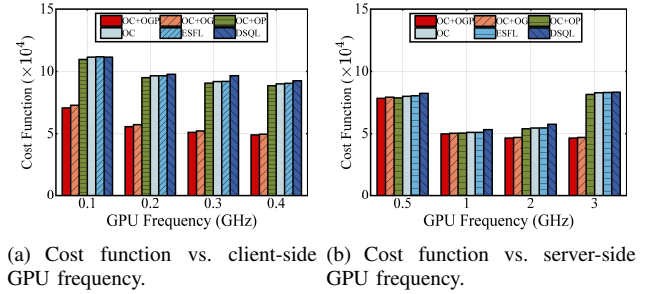


Fig. 5. The impact of GPU frequency on cost function.

ing its effectiveness under different communication resource conditions. Fig. 5 illustrates the impact of the maximum allowable GPU frequency on the cost function. In Fig. 5(a), increasing the maximum allowable client-side GPU frequency continuously reduces the cost for all compared methods, which indicates that, within the tested range, enlarging the feasible client-side frequency set is beneficial for cost reduction. Fig. 5(b) shows that enlarging the maximum allowable server-side GPU frequency does not necessarily reduce the cost. For the proposed **OC+OGP** and **OC+OG**, the cost remains unchanged once the feasible range already includes the server-side GPU frequency that gives the lowest cost, since the proposed method chooses the cost-minimizing frequency instead of simply using the maximum available frequency. By contrast, the baseline schemes incur higher cost at large server-side frequency values because they operate at the maximum allowable GPU frequency.

In edge networks, computing resources and channel conditions may fluctuate during training, resulting in a mismatch between the measured parameters used for optimization and the actual operating conditions. To evaluate the impact of such

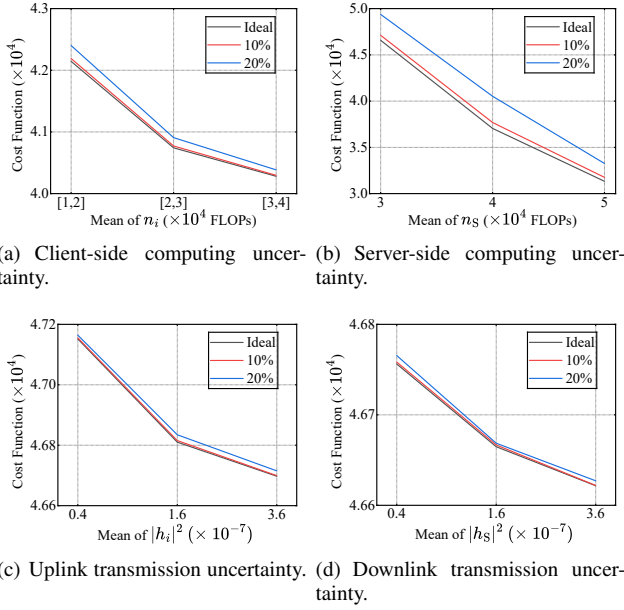


Fig. 6. The impact of network resource fluctuation on cost function.

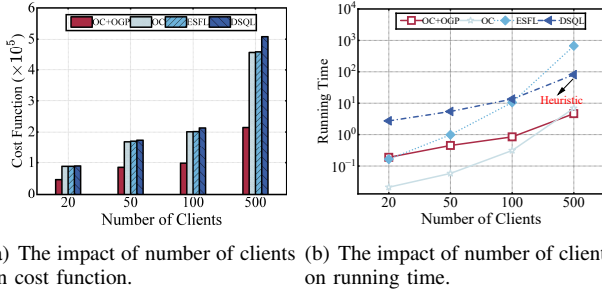
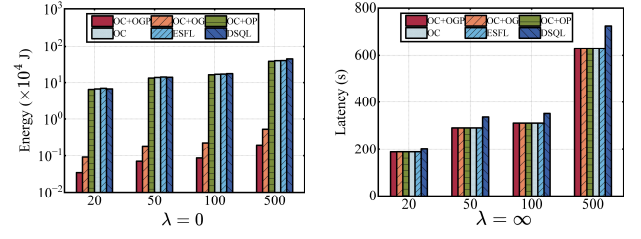


Fig. 7. The impact of number of clients on cost function and running time.

uncertainty, we model the computing capabilities and average channel power gains as Gaussian random perturbations around their nominal values, with different coefficients of variation (CV) [43], [44]. Specifically, we consider uncertainty in four key parameters: n_i , n_S , $|h_i|^2$, and $|h_S|^2$, where n_i and n_S denote the numbers of GPU FLOPs per cycle of client i and the edge server, respectively, and $|h_i|^2$ and $|h_S|^2$ denote the average uplink and downlink channel power gains, respectively. In our experiments, the CV is set to 10% and 20% to represent moderate and high uncertainty levels, respectively. For each CV level, we repeat the simulation over 200 random realizations. Fig. 6 shows that the objective value increases as the CV grows, due to the mismatch between the optimized parameters and the actual conditions. Nevertheless, the proposed method consistently maintains competitive performance, demonstrating good robustness under practical resource uncertainty.

C. Running Time Comparison

Fig. 7 illustrates the impact of the number of clients on both the cost function and the running time. As shown in Fig. 7(a), the cost function generally increases as the number of clients grows for all methods compared. The proposed

Fig. 8. Performance comparison under $\lambda = 0$ and $\lambda = \infty$.

method **OC+OGP** achieves a lower cost than baselines across all the numbers of clients tested. Moreover, the proposed **OC** method achieves the same cost as ESFL, while consistently outperforming DSQ. Fig. 7(b) further shows that the proposed efficient **OC+OGP** and **OC** methods require lower running time compared with ESFL and DSQ. To sum up, the results verify the theoretical analysis that the proposed **OC** attains the optimal solution with polynomial-time complexity. In addition, it shows that **OC+OGP** remains polynomial-time as the number of clients increases, which indicates that the proposed method is able to obtain high-quality approximate solutions efficiently.

D. Impact of Hyperparameter λ

Fig. 8 further examines the two extreme cases. When $\lambda = 0$, i.e., under energy-only optimization, the proposed **OC+OGP** achieves the lowest energy among all compared methods. When $\lambda = \infty$, i.e., under latency-only optimization, **OC+OGP** achieves nearly the same latency as **OC+OG**, **OC+OP**, **OC**, and ESFL, while remaining lower than DSQ. This is because the latency-only objective drives the optimized GPU frequencies and transmit powers to their maximum feasible values, so the additional resource-control degrees of freedom in **OC+OGP**, **OC+OG**, and **OC+OP** do not further reduce latency beyond the cut-layer optimization. These results demonstrate that the proposed method can adapt to different optimization preferences and maintain favorable performance under both energy-oriented and latency-oriented settings.

VIII. CONCLUSION

In this paper, we investigated the joint optimization of model splitting and resource allocation for SFL in resource-constrained edge systems. We formulated the resulting design as a hybrid optimization problem that captures the coupled effects of computation, communication, and model-splitting decisions. For the special model-splitting case, we developed an optimal polynomial-time solution. For the general joint optimization problem, we reformulated it as a two-dimensional master problem and developed a computationally efficient approximation method with explicit theoretical guarantees. Numerical results demonstrated the effectiveness of the proposed approach and its advantage over existing baselines. Overall, the proposed framework provides an efficient method for resource optimization in SFL under the energy-latency tradeoff.

REFERENCES

- [1] W. Wu, M. Li, K. Qu, C. Zhou, X. Shen, W. Zhuang, X. Li, and W. Shi, "Split learning over wireless networks: Parallel design and resource management," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 4, pp. 1051–1066, Feb. 2023.
- [2] W. Fan, P. Chen, X. Chun, and Y. Liu, "MADRL-based model partitioning, aggregation control, and resource allocation for cloud-edge-device collaborative split federated learning," *IEEE Transactions on Mobile Computing*, vol. 24, no. 6, pp. 5324–5341, May 2025.
- [3] W. Wei, Z. Lin, T. Li, X. Li, and X. Chen, "Pipelining split learning in multi-hop edge networks," *IEEE Transactions on Mobile Computing*, pp. 1–14, 2026.
- [4] Z. Wang, K. Huang, and Y. C. Eldar, "Spectrum breathing: Protecting over-the-air federated learning against interference," *IEEE Trans. Wireless Commun.*, vol. 23, no. 8, pp. 10058–10071, 2024.
- [5] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 6, pp. 3820–3860, Dec. 2025.
- [6] NVIDIA, "NVIDIA Jetson Xavier," 2019. [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-series/>
- [7] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split Learning for Health: Distributed Deep Learning Without Sharing Raw Patient Data," in *ICLR workshop on AI for social good*, New Orleans, LA, USA, Apr. 2019, pp. 1–7.
- [8] Z. Lin, G. Qu, X. Chen, and K. Huang, "Split Learning in 6G Edge Networks," *IEEE Wireless Communications*, vol. 31, no. 4, pp. 170–176, Aug. 2024.
- [9] Z. Yao, J. Qi, Y. Xu, Y. Liao, H. Xu, and L. Wang, "Pairingfl: Efficient federated learning with model splitting and client pairing," *IEEE Transactions on Networking*, vol. 33, no. 4, pp. 1811–1825, May 2025.
- [10] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, May 2020.
- [11] J. Konečný, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.
- [12] G. Lan, X.-Y. Liu, Y. Zhang, and X. Wang, "Communication-efficient federated learning for resource-constrained edge devices," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 1, pp. 210–224, Aug. 2023.
- [13] Z. Lin, G. Qu, W. Wei, X. Chen, and K. K. Leung, "AdaptSFL: Adaptive split federated learning in resource-constrained edge networks," *IEEE Transactions on Networking*, vol. 33, no. 6, pp. 2993–3008, Jun. 2025.
- [14] Z. Lin, W. Wei, Z. Chen, C.-T. Lam, X. Chen, Y. Gao, and J. Luo, "Hierarchical split federated learning: Convergence analysis and system optimization," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9352–9367, Apr. 2025.
- [15] W. Wei, Z. Lin, X. Liu, H. Du, D. Niyato, and X. Chen, "Optimizing split federated learning with unstable client participation," *IEEE Transactions on Mobile Computing*, pp. 1–15, Mar. 2026.
- [16] G. Qu, T. Li, Q. Chen, X. Chen, and S. Zhou, "Slide: Simultaneous model downloading and inference at the wireless network edge," *IEEE Transactions on Mobile Computing*, pp. 1–12, 2026.
- [17] G. Zhu, Y. Deng, X. Chen, H. Zhang, Y. Fang, and T. F. Wong, "Esfl: Efficient split federated learning over resource-constrained heterogeneous wireless devices," *IEEE Internet of Things Journal*, vol. 11, no. 16, pp. 27 153–27 166, Aug. 2024.
- [18] W. Wei, J. Wang, J. Du, Z. Fang, C. Jiang, and Y. Ren, "Underwater differential game: Finite-time target hunting task with communication delay," in *IEEE International Conference on Communications (ICC)*, Seoul, Korea, May, 2022, pp. 3989–3994.
- [19] W. Wei, J. Wang, J. Du, Z. Fang, Y. Ren, and C. L. P. Chen, "Differential game-based deep reinforcement learning in underwater target hunting task," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 1, pp. 462–474, Jan., 2025.
- [20] W. Wei, J. Wang, Z. Fang, J. Chen, Y. Ren, and Y. Dong, "3U: Joint design of UAV-USV-UUV networks for cooperative target hunting," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 3, pp. 4085–4090, Mar. 2023.
- [21] C. Guo, F. Zhou, L. Feng, and W. Li, "Hierarchical multiple split federated learning for low-carbon resource-constrained user equipment," *IEEE Internet of Things Journal*, vol. 11, no. 24, pp. 39 127–39 141, Dec. 2024.
- [22] C. Meng, M. Tang, and V. W. S. Wong, "ASFL: An adaptive model splitting and resource allocation framework for split federated learning," *arXiv preprint arXiv:2603.04437*, 2026.
- [23] M. Wu, R. Yang, X. Huang, Y. Wu, J. Kang, and S. Xie, "Joint optimization of model partition and resource allocation for split federated learning over vehicular edge networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 10, pp. 15 860–15 865, Oct. 2024.
- [24] X. Wang, S. Song, Z. Zhang, X. Hou, Z. Li, T. Xing, and X.-P. Zhang, "Split federated learning for resource-constrained edge computing networks," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11 001–11 013, Nov. 2025.
- [25] Z. Wang, Z. Zhang, J. Wang, C. Jiang, W. Wei, and Y. Ren, "AUV-assisted node repair for iout relying on multiagent reinforcement learning," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4139–4151, 2024.
- [26] Q. Chen, X. Chen, and K. Huang, "SiftMoE: Similarity-aware energy-efficient expert selection for wireless distributed moe inference," *arXiv preprint arXiv:2603.23888*, 2026.
- [27] R. Drechsler, C. A. Metz, and C. Plump, "Energy-efficient CNN inferencing on GPUs with dynamic frequency scaling," in *International Conference on Innovations in Data Analytics*, Kolkata, India, Nov. 2023, pp. 375–389.
- [28] P. J. Maliakel, S. Ilager, and I. Brandic, "Characterizing LLM inference energy-performance tradeoffs across workloads and GPU scaling," *arXiv preprint arXiv:2501.08219*, Feb. 2026.
- [29] A. K. Kakolyris, D. Masouros, P. Vavaroutsos, S. Xydis, and D. Soudris, "throttLL'eM: Predictive GPU throttling for energy efficient LLM inference serving," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Las Vegas, NV, USA, Mar. 2025, pp. 1363–1378.
- [30] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*, Boston, MA, USA, Apr. 2023, p. 119 – 139.
- [31] S. Choi, I. Koo, J. Ahn, M. Jeon, and Y. Kwon, "ENVPIPE: Performance-preserving DNN training framework for saving energy," in *Proceedings of the USENIX Annual Technical Conference*, Boston, MA, USA, Jul. 2023, p. 851 – 864.
- [32] Z. Jia, L. N. Bhuyan, and D. Wong, "PCCL: Energy-efficient LLM training with power-aware collective communication," in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, Milan, Italy, Nov. 2024, pp. 84–91.
- [33] J.-W. Chung, Y. Gu, I. Jang, L. Meng, N. Bansal, and M. Chowdhury, "Reducing energy bloat in large model training," in *Proceedings of the ACM Symposium on Operating Systems Principles*, New York, NY, USA, Nov. 2024, p. 144–159.
- [34] Q. Zeng, Y. Du, K. Huang, and K. K. Leung, "Energy-efficient resource management for federated edge learning with CPU-GPU heterogeneous computing," *IEEE Transactions on Wireless Communications*, vol. 20, no. 12, pp. 7947–7962, Dec. 2021.
- [35] A. E. Bryson, *Applied optimal control: optimization, estimation and control*. Routledge, 2018.
- [36] K. Gokbayrak and C. Cassandras, "A hierarchical decomposition method for optimal control of hybrid systems," in *Proceedings of IEEE Conference on Decision and Control*, Sydney, NSW, Australia, Dec. 2000, pp. 1816–1821.
- [37] L. Deng, "The MNIST database of handwritten digit images for machine learning research," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, Nov. 2012.
- [38] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *arXiv preprint arXiv:1512.03385*, 2015.
- [40] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, Apr. 2015.
- [41] NVIDIA, "Solving entry-level edge AI challenges with NVIDIA Jetson Orin Nano," <https://developer.nvidia.com/blog/solving-entry-level-edge-ai-challenges-with-nvidia-jetson-orin-nano/>, accessed: 2026-05-07.
- [42] —, "NVIDIA system management interface (nvidia-smi)," <https://docs.nvidia.com/deploy/nvidia-smi/>, accessed: 2026-05-07.
- [43] W. J. Robinson M., F. Esposito, and M. A. Zuluaga, "DTS: A simulator to estimate the training time of distributed deep neural networks," in *International Symposium on Modeling, Analysis, and Simulation of*

- Computer and Telecommunication Systems*, Nice, France, Mar. 2023, pp. 17–24.
- [44] Y. Yoo and S. Jung, “Modeling forecast errors for microgrid operation using Gaussian process regression,” *Scientific Reports*, vol. 14, no. 1, p. 2166, Jan. 2024.